

Análise Comparativa: Método Sequencial vs Programação Linear v2.0

Sistema SIVIRA - Scheduling de Produção em Padaria Industrial

Sistema SIVIRA
www.sivira.com

14 de Novembro de 2025

Resumo

Este documento apresenta uma análise comparativa rigorosa entre dois métodos de *scheduling* de produção implementados no sistema SIVIRA: (1) Método Sequencial baseado em heurística *greedy* com *backward scheduling* e ordenação EDD; e (2) Programação Linear v2.0 utilizando Mixed Integer Programming (MIP) com solver SCIP. Os resultados empíricos com 13 pedidos reais mostram que o método sequencial atinge 84.6% de taxa de sucesso (11/13 pedidos) com *makespan* de 29h em 0.42s, enquanto o PL v2.0 atinge 69.2% (9/13 pedidos) com *makespan* de 73h em 0.18s. Apesar da garantia de otimalidade do PL, o método sequencial apresenta melhor desempenho prático devido ao espaço de busca mais amplo e função objetivo implicitamente multi-critério. O documento discute este paradoxo, analisa trade-offs fundamentais e propõe melhorias para ambos os métodos.

Sumário

1	Introdução e Contexto	2
1.1	Problema de Scheduling de Produção	2
1.2	Características do Problema	2
1.3	Abordagens Implementadas	2
2	Formulação Matemática do PL v2.0	3
2.1	Notação e Conjuntos	3
2.2	Variáveis de Decisão	3
2.3	Função Objetivo	3
2.4	Restrições	4
2.4.1	Restrição de Unicidade de Janela	4
2.4.2	Restrições de Tempo Máximo de Espera	4
2.4.3	Restrições de Conflitos Temporais	4
2.5	Modelo Completo	5

2.6	Estatísticas do Modelo	5
3	Método Sequencial (Backward Scheduling)	5
3.1	Descrição do Algoritmo	5
3.2	Pseudo-código	6
3.3	Backward Scheduling Formal	6
3.4	Análise de Complexidade	6
3.4.1	Complexidade Temporal	6
3.4.2	Complexidade Espacial	7
4	Comparação Empírica	7
4.1	Conjunto de Dados	7
4.2	Resultados Quantitativos	7
4.3	Análise de Makespan	8
4.4	Análise de Tempo Computacional	8
5	Discussão e Trade-offs	8
5.1	Paradoxo da Otimalidade	8
5.2	Trade-offs Fundamentais	9
5.2.1	Otimalidade vs Eficiência	9
5.2.2	Makespan vs Número de Pedidos	9
5.2.3	Precisão Temporal vs Complexidade	9
6	Propostas de Melhoria	9
6.1	Melhorias para PL v2.0	9
6.1.1	Função Objetivo Multi-critério	9
6.1.2	Geração Adaptativa de Janelas	10
6.1.3	Método Híbrido (Warm Start)	10
6.2	Melhorias para Método Sequencial	10
6.2.1	Heurísticas de Ordenação Alternativas	10
6.2.2	Look-ahead Local	10
7	Conclusões e Recomendações	11
7.1	Principais Descobertas	11
7.2	Recomendações de Uso	11
7.3	Trabalhos Futuros	11
7.3.1	Curto Prazo	11
7.3.2	Médio Prazo	11
7.3.3	Longo Prazo	11
7.4	Mensagem Final	12
A	Glossário	12
B	Dados do Experimento	13

1 Introdução e Contexto

1.1 Problema de Scheduling de Produção

O sistema SIVIRA resolve um problema de **Job Shop Scheduling** com características específicas de uma padaria industrial. Este problema pode ser formalmente classificado como:

$$Jm \mid prec, r_j, d_j \mid C_{max} \text{ ou } \sum U_j \quad (1)$$

onde:

- Jm : Job Shop com m máquinas
- $prec$: Restrições de precedência entre atividades
- r_j : Datas de liberação (release dates)
- d_j : Prazos (due dates)
- C_{max} : Makespan (minimizar)
- $\sum U_j$: Número de trabalhos atrasados (minimizar)

Este é um problema **NP-difícil** [1], o que justifica o uso de heurísticas eficientes além de métodos exatos.

1.2 Características do Problema

O problema apresenta as seguintes características:

1. **Múltiplos pedidos** com diferentes produtos
2. **Atividades sequenciais** com precedências estritas
3. **Recursos compartilhados** (equipamentos com capacidade limitada)
4. **Restrições temporais**:
 - Deadlines por pedido
 - Tempo máximo de espera entre atividades
 - Janelas de produção (horário comercial)
5. **Objetivo**: Maximizar número de pedidos atendidos

1.3 Abordagens Implementadas

Duas abordagens foram desenvolvidas e comparadas:

Método Sequencial Heurística *greedy* com complexidade $\mathcal{O}(n \log n + n^2)$

Programação Linear v2.0 Modelo MIP com complexidade NP-completo

2 Formulação Matemática do PL v2.0

2.1 Notação e Conjuntos

Definimos os seguintes conjuntos e parâmetros:

Símbolo	Descrição	Cardinalidade
P	Conjunto de pedidos	$ P = n$
J_p	Janelas viáveis para pedido $p \in P$	$ J_p \leq j_{max}$
A_p	Atividades do pedido p	$ A_p = m_p$
E	Conjunto de equipamentos	$ E = e$
T	Conjunto de slots temporais	$ T = t$

Tabela 1: Conjuntos básicos do modelo

Parâmetros de configuração:

- $j_{max} = 15$: Número máximo de janelas por pedido
- $\Delta t = 30$: Resolução temporal em minutos
- $t_{timeout} = 600$: Timeout do solver em segundos

2.2 Variáveis de Decisão

A variável principal do modelo é binária:

$$x_{p,j} \in \{0, 1\} \quad \forall p \in P, j \in J_p \quad (2)$$

Interpretação:

$$x_{p,j} = \begin{cases} 1 & \text{se pedido } p \text{ usa janela } j \\ 0 & \text{caso contrário} \end{cases} \quad (3)$$

O número total de variáveis é:

$$|X| = \sum_{p \in P} |J_p| \quad (4)$$

Exemplo prático: Para $n = 13$ pedidos com $|J_p| = 9$ janelas cada, temos $|X| = 117$ variáveis.

2.3 Função Objetivo

O objetivo é maximizar o número de pedidos atendidos:

$$\max \sum_{p \in P} y_p \quad (5)$$

onde y_p indica se o pedido p foi atendido:

$$y_p = \begin{cases} 1 & \text{se } \exists j \in J_p : x_{p,j} = 1 \\ 0 & \text{caso contrário} \end{cases} \quad (6)$$

Como a restrição de unicidade (Equação 8) garante $\sum_j x_{p,j} \leq 1$, podemos linearizar:

$$\max \sum_{p \in P} \sum_{j \in J_p} x_{p,j} \quad (7)$$

2.4 Restrições

2.4.1 Restrição de Unicidade de Janela

Cada pedido pode usar no máximo uma janela temporal:

$$\sum_{j \in J_p} x_{p,j} \leq 1 \quad \forall p \in P \quad (8)$$

Esta restrição gera $|P| = n$ inequações lineares.

2.4.2 Restrições de Tempo Máximo de Espera

Para pedidos com atividades sucessoras que têm tempo máximo de espera $\tau_{max}(a_i)$:

$$gap(a_i, a_{i+1}) \leq \tau_{max}(a_i) \quad \forall i \in \{1, \dots, m-1\} \quad (9)$$

onde:

$$gap(a_i, a_{i+1}) = inicio(a_{i+1}) - fim(a_i) \quad (10)$$

Caso especial ($\tau_{max} = 0$, atividades contíguas):

$$inicio(a_{i+1}) = fim(a_i) \quad (11)$$

2.4.3 Restrições de Conflitos Temporais

Duas janelas j_1 e j_2 se **sobrepõem** se:

$$overlap(j_1, j_2) \iff \begin{cases} inicio_{j_1} < fim_{j_2} \\ inicio_{j_2} < fim_{j_1} \end{cases} \quad (12)$$

Para cada par de janelas que se sobrepõem:

$$x_{p_1, j_1} + x_{p_2, j_2} \leq 1 \quad \forall p_1 \neq p_2, j_1 \in J_{p_1}, j_2 \in J_{p_2} : overlap(j_1, j_2) \quad (13)$$

O número de restrições de conflito no pior caso é:

$$R_{conflitos} = \binom{|J_{total}|}{2} = \frac{|J_{total}| \times (|J_{total}| - 1)}{2} \quad (14)$$

2.5 Modelo Completo

O modelo completo de programação linear inteira é:

$$\begin{aligned}
 \max \quad & \sum_{p \in P} \sum_{j \in J_p} x_{p,j} \\
 \text{s.a.} \quad & \sum_{j \in J_p} x_{p,j} \leq 1 \quad \forall p \in P \\
 & x_{p_1,j_1} + x_{p_2,j_2} \leq 1 \quad \forall (p_1, j_1), (p_2, j_2) : \text{overlap}(j_1, j_2) \\
 & x_{p,j} \in \{0, 1\} \quad \forall p \in P, j \in J_p
 \end{aligned} \tag{15}$$

2.6 Estatísticas do Modelo

Para o conjunto de dados com 13 pedidos, o modelo apresenta as seguintes características:

Métrica	Otimizado	Antigo
Variáveis	117	65
Restrições Totais	1,321	513
Unicidade	13	13
Tempo Máx Espera	0	0
Equipamentos	0	0
Conflitos Temporais	1,308	500
Status Solver	OPTIMAL	OPTIMAL
Tempo Resolução	0.18s	0.04s

Tabela 2: Estatísticas do modelo PL v2.0

3 Método Sequencial (Backward Scheduling)

3.1 Descrição do Algoritmo

O método sequencial é uma heurística *greedy* baseada em três etapas:

1. **Ordenação** dos pedidos por deadline (EDD - Earliest Due Date)
2. **Backward scheduling** para cada pedido
3. **Alocação sequencial** de equipamentos

3.2 Pseudo-código

Algorithm 1 Scheduling Sequencial

Require: Lista de pedidos $P = \{p_1, p_2, \dots, p_n\}$

Ensure: Schedule S ou FALHA

```

1:  $P_{ordenado} \leftarrow \text{SORT}(P, \text{key} = \lambda p : p.\text{deadline})$ 
2: for cada pedido  $p$  em  $P_{ordenado}$  do
3:    $A_p \leftarrow \text{CRIAR\_ATIVIDADES}(p)$ 
4:    $\text{inicio}_{corrente} \leftarrow p.\text{deadline}$ 
5:   for cada atividade  $a$  em  $\text{REVERSO}(A_p)$  do
6:      $\text{fim}_{desejado} \leftarrow \text{inicio}_{corrente}$ 
7:      $\text{inicio}_{desejado} \leftarrow \text{fim}_{desejado} - a.\text{duracao}$ 
8:      $\text{sucesso} \leftarrow \text{ALOCAR\_EQUIPAMENTOS}(a, \text{inicio}_{desejado}, \text{fim}_{desejado})$ 
9:     if não  $\text{sucesso}$  then
10:      rejeitar pedido  $p$ 
11:      continuar com próximo pedido
12:    end if
13:     $\text{inicio}_{corrente} \leftarrow \text{inicio}_{real}$  de  $a$ 
14:  end for
15:   $\text{REGISTRAR\_SUCESSO}(p)$ 
16: end for
17: return schedule  $S$ 

```

3.3 Backward Scheduling Formal

Para um pedido com atividades $\{a_1, a_2, \dots, a_m\}$ e deadline d :

$$\text{fim}(a_m) \leq d \quad (16)$$

$$\text{inicio}(a_m) = \text{fim}(a_m) - \text{duracao}(a_m) \quad (17)$$

$$\text{fim}(a_{m-1}) = \text{inicio}(a_m) - \text{gap}(a_{m-1}, a_m) \quad (18)$$

$$\text{inicio}(a_{m-1}) = \text{fim}(a_{m-1}) - \text{duracao}(a_{m-1}) \quad (19)$$

$$\vdots$$

$$\text{inicio}(a_1) = \text{fim}(a_1) - \text{duracao}(a_1) \quad (20)$$

3.4 Análise de Complexidade

3.4.1 Complexidade Temporal

1. Ordenação EDD: $\mathcal{O}(n \log n)$
2. Para cada pedido:
 - Criar atividades: $\mathcal{O}(m)$
 - Backward scheduling: $\mathcal{O}(m)$

- Para cada atividade:
 - Buscar equipamento: $\mathcal{O}(e)$
 - Buscar slot: $\mathcal{O}(s)$
 - Alocar: $\mathcal{O}(1)$

Complexidade total:

$$T_{seq}(n) = \mathcal{O}(n \log n) + \sum_{i=1}^n \mathcal{O}(m_i \times (e + s_i)) \quad (21)$$

No pior caso (todos executados):

$$T_{seq}(n) = \mathcal{O}(n \log n + n^2) \quad (22)$$

Classe de complexidade: Polinomial $\mathcal{O}(n^2)$

3.4.2 Complexidade Espacial

$$S_{seq}(n) = \mathcal{O}(n \times m \times e) \approx \mathcal{O}(n) \quad (23)$$

assumindo m e e constantes.

4 Comparação Empírica

4.1 Conjunto de Dados

O dataset consiste em 13 pedidos reais de uma padaria industrial, cobrindo o período de 28/12 07:00 até 31/12 08:00 (72 horas). Os pedidos incluem pães com fermentação longa (19-27h) e salgados de produção rápida (2-5h).

4.2 Resultados Quantitativos

Métrica	Sequencial	PL v2.0	Diferença
Pedidos Atendidos	11/13	9/13	-2 pedidos
Taxa de Sucesso	84.6%	69.2%	-15.4 p.p.
Tempo Resolução	0.42s	0.18s	-57%
Makespan	29h	73h	+151%
Pedidos Rejeitados	P2, P3	P2, P3, P11, P13	+2
Complexidade	N/A	117 vars, 1321 const	–
Status	Completo	OPTIMAL	–
Garantias	Nenhuma	Otimalidade	✓

Tabela 3: Comparação quantitativa dos métodos

4.3 Análise de Makespan

O método sequencial produz um *schedule* $2.5\times$ mais compacto (29h vs 73h), resultando em melhor utilização de recursos:

Equipamento	Sequencial	PL v2.0	Diferença
Forno 1	78%	52%	-26 p.p.
Misturador 1	85%	48%	-37 p.p.
Modeladora 1	92%	61%	-31 p.p.
Fermentador 1	100%	73%	-27 p.p.

Tabela 4: Utilização de recursos

4.4 Análise de Tempo Computacional

Fase	Sequencial	PL v2.0
Setup	0.15s	0.09s
Core (resolução)	0.25s	0.07s
Pós-processamento	0.02s	0.02s
Total	0.42s	0.18s

Tabela 5: Perfil de tempo de execução

5 Discussão e Trade-offs

5.1 Paradoxo da Otimalidade

Observamos um resultado contra-intuitivo: o método PL v2.0, apesar de matematicamente ótimo, apresentou **performance inferior** ao sequencial (69.2% vs 84.6%).

Explicação do Paradoxo

1. **Otimalidade relativa:** PL é ótimo apenas *dentro do espaço de soluções* definido pelas janelas geradas
2. **Função objetivo limitada:** Maximizar número de pedidos não considera makespan
3. **Discretização temporal:** Resolução de 30min pode perder soluções contínuas
4. **Exploração do espaço:** Sequencial testa todas as posições possíveis (busca em resolução de 1min)

5.2 Trade-offs Fundamentais

Identificamos três trade-offs principais:

5.2.1 Otimalidade vs Eficiência

- **Sequencial:** Eficiente ($\mathcal{O}(n^2)$) mas sem garantias
- **PL v2.0:** Garantias formais mas NP-completo
- **Gap:** Espaço para métodos híbridos

5.2.2 Makespan vs Número de Pedidos

Existe um trade-off fundamental não capturado pela função objetivo atual:

- **Sequencial:** 11 pedidos em 29h (makespan compacto)
- **PL v2.0:** 9 pedidos em 73h (makespan esparso)

5.2.3 Precisão Temporal vs Complexidade

Resolução	Variáveis	Tempo	Qualidade
60 min	65	0.04s	38.5% (5/13)
30 min	117	0.18s	69.2% (9/13)
15 min (proj)	~210	~2.5s	~75%
5 min (proj)	~630	~45s	~80%

Tabela 6: Trade-off resolução temporal vs complexidade

6 Propostas de Melhoria

6.1 Melhorias para PL v2.0

6.1.1 Função Objetivo Multi-critério

Proposta de nova função objetivo:

$$\max \left(w_1 \cdot \sum_p y_p - w_2 \cdot C_{max} - w_3 \cdot \sum_p L_p \right) \quad (24)$$

onde:

- $w_1 = 100$: Peso para número de pedidos
- $w_2 = 0.1$: Peso para makespan
- $w_3 = 10$: Peso para atrasos ($L_p = \max(0, C_p - d_p)$)

Impacto esperado: Reduzir makespan de 73h para ~35h mantendo ou melhorando pedidos atendidos.

6.1.2 Geração Adaptativa de Janelas

Estratégia diferenciada por tipo de pedido:

$$pontos = \begin{cases} [0.0, 0.05, 0.1, 0.15, 0.2] & \text{se } duracao_p > 20h \\ [0.0, 0.125, \dots, 0.875, 1.0] & \text{caso contrário} \end{cases} \quad (25)$$

Impacto esperado: Melhorar atendimento de pedidos longos, aumentando taxa de 69% para ~80%.

6.1.3 Método Híbrido (Warm Start)

Algoritmo proposto:

Algorithm 2 Método Híbrido

```

1:  $solucao_{seq} \leftarrow \text{SequencialRapido}()$ 
2:  $warm\_start \leftarrow \text{ConverterParaPL}(solucao_{seq})$ 
3:  $solucao_{pl} \leftarrow \text{PL\_v2.0}(warm\_start)$ 
4: return  $\max(solucao_{seq}, solucao_{pl})$ 

```

Impacto esperado: Garantir pelo menos performance do sequencial, com possibilidade de melhorias via PL.

6.2 Melhorias para Método Sequencial

6.2.1 Heurísticas de Ordenação Alternativas

Além de EDD, testar:

- **WSPT** (Weighted Shortest Processing Time):

$$priority_p = \frac{w_p}{p_p} \quad (26)$$

- **ATC** (Apparent Tardiness Cost):

$$priority_p = \frac{w_p}{p_p} \exp \left(-\frac{\max(d_p - t - p_p, 0)}{kp_{avg}} \right) \quad (27)$$

6.2.2 Look-ahead Local

Antes de alocar pedido p :

1. Simular impacto em próximos 3-5 pedidos
2. Se conflito severo detectado, testar ordem alternativa
3. Usar busca local limitada

7 Conclusões e Recomendações

7.1 Principais Descobertas

1. Ambos os métodos são viáveis para produção real
2. Método "ótimo"(PL) teve performance inferior ao heurístico devido a espaço de busca restrito
3. PL v2.0 é 57% mais rápido (0.18s vs 0.42s)
4. Sequencial produz makespan $2.5\times$ mais compacto
5. Existe complementaridade entre os métodos

7.2 Recomendações de Uso

Cenário	Método	Justificativa
$n \leq 30$ pedidos	PL v2.0	Rápido + otimalidade
$n > 50$ pedidos	Sequencial	PL não escala
Deadline apertado	Sequencial	Makespan compacto
Pedidos longos ($>20h$)	Sequencial	PL rejeita
Tempo real ($<1s$)	PL v2.0	Mais rápido
Pesquisa/análise	Ambos	Comparar

Tabela 7: Recomendações de uso por cenário

7.3 Trabalhos Futuros

7.3.1 Curto Prazo

- Implementar função objetivo multi-critério
- Desenvolver geração adaptativa de janelas
- Criar método híbrido (Seq + PL)

7.3.2 Médio Prazo

- Benchmark extensivo (50+ instâncias)
- Análise de sensibilidade dos parâmetros
- Decomposição temporal para escalabilidade

7.3.3 Longo Prazo

- Aprendizado de máquina para ordenação
- Otimização multi-objetivo (Pareto)
- Scheduling dinâmico e re-scheduling

7.4 Mensagem Final

Conclusão Principal

“A escolha entre métodos depende do contexto. Não existe ‘melhor método absoluto’.”

O valor real está em **entender os trade-offs** e **escolher conscientemente** baseado nas necessidades específicas de cada situação. Completude de modelagem não garante superioridade prática.

Referências

- [1] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems* (5th ed.). Springer.
- [2] Brucker, P. (2007). *Scheduling Algorithms* (5th ed.). Springer.
- [3] Wolsey, L. A., & Nemhauser, G. L. (1999). *Integer and Combinatorial Optimization*. Wiley.
- [4] Bertsimas, D., & Weismantel, R. (2005). *Optimization over Integers*. Dynamic Ideas.
- [5] Google OR-Tools (2024). *Google Optimization Tools*. <https://developers.google.com/optimization>

A Glossário

Job Shop Scheduling Problema de alocar trabalhos (jobs) a máquinas ao longo do tempo

Makespan Tempo total do schedule (do início do primeiro ao fim do último trabalho)

Deadline Prazo máximo para completar um trabalho

EDD Earliest Due Date - Heurística que ordena por deadline crescente

Backward Scheduling Agendar de trás para frente (do deadline para o início)

Janela Temporal Intervalo $[inicio, fim]$ onde um trabalho pode ser executado

MIP Mixed Integer Programming - Programação inteira mista

Branch-and-bound Algoritmo de busca em árvore para resolver MIP

SCIP Solving Constraint Integer Programs - Solver open-source

B Dados do Experimento

ID	Produto	Qtd	Deadline	Duração
P1	Pão Francês	420	31/12 07:00	~6h
P2	Pão Hambúrguer	120	31/12 07:00	~27h
P3	Pão de Forma	20	31/12 07:00	~19h
P4	Pão Baguete	20	31/12 07:00	~4h
P5	Pão Trança	15	31/12 07:00	~4h
P6	Coxinha Frango	15	31/12 08:00	~3h
P7	Coxinha C. Sol	10	31/12 08:00	~2h
P8	Coxinha Camarão	12	31/12 08:00	~3h
P9	Coxinha Queijos	12	31/12 08:00	~3h
P10	Folhado Frango	10	31/12 08:00	~4h
P11	Folhado C. Sol	10	31/12 08:00	~3h
P12	Folhado Camarão	10	31/12 08:00	~5h
P13	Folhado Queijos	5	31/12 08:00	~3h

Tabela 8: Dataset completo do experimento